UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

SEDE SANTO DOMINGO DE LOS TSÁCHILAS

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

CARRERA DE INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN

ASIGNATURA:

Modelos Discretos Para la Ingeniería

TEMA:

Proyecto Unidad 1

Complejidad Algorítmica en el Proyecto

Royal Casino

DOCENTE:

Ing. Javier Cevallos Farías

ESTUDIANTES:

Jeffry David Vera Borja

Darlin Jair Demera Yánez

NIVEL — NRC:

Tercero — 29532

PERÍODO:

S-I ABR 26 – AGO 26

FECHA DE ENTREGA:

17/06/2026

TABLA DE CONTENIDOS

1. Introducción 3
2. Objetivos 4
3. Descripción del Problema o Caso Aplicado 5
4. Temas de Lógica Utilizados 6
5. Desarrollo del Sistema — Descripción Paso a Paso 11
6. Ejemplos Resueltos 18
7. Conclusiones 23
8. Recomendaciones 24
9. Referencias 25
Anexos 26

1. INTRODUCCIÓN

En el desarrollo de software, no basta con que un programa funcione correctamente; también es importante que lo haga de manera eficiente. La complejidad algorítmica permite analizar el rendimiento de los algoritmos y estudiar cómo varía su tiempo de ejecución cuando aumenta la cantidad de datos que deben procesar. Para ello, se emplea la notación Big-O, una herramienta fundamental que clasifica los algoritmos según su comportamiento y escalabilidad.

En el presente trabajo se analiza la complejidad algorítmica de diferentes funciones implementadas en el proyecto Royal Casino, una aplicación interactiva que simula juegos de casino como ruleta y blackjack. A través de pruebas experimentales realizadas con la función performance.now(), se midieron los tiempos de ejecución de tres procesos representativos del sistema: el control de acceso de usuarios, el dibujo de la ruleta y el cálculo de puntajes en blackjack.

El objetivo de este análisis es comparar los resultados obtenidos experimentalmente con las predicciones teóricas de la complejidad algorítmica, identificando funciones de tiempo constante O(1) y funciones de tiempo lineal O(n). De esta manera, se busca comprender la importancia de evaluar la eficiencia de los algoritmos para garantizar un mejor rendimiento y una adecuada escalabilidad en aplicaciones informáticas reales.

2. OBJETIVOS

2.1 Objetivo General

Evaluar el rendimiento de los algoritmos utilizados en el proyecto Royal Casino mediante el estudio de su complejidad temporal y la realización de pruebas experimentales, con el propósito de determinar su eficiencia, escalabilidad y correspondencia con los modelos teóricos de complejidad algorítmica O(1) y O(n).

2.2 Objetivos Específicos

- Identificar las funciones principales del proyecto Royal Casino que serán analizadas desde el punto de vista de la complejidad algorítmica.
- Medir los tiempos de ejecución de las funciones seleccionadas utilizando la herramienta performance.now() para obtener datos experimentales precisos.
- Determinar el comportamiento temporal de las funciones de control de acceso, dibujo de la ruleta y cálculo de puntaje en blackjack al variar el tamaño de entrada.
- Comparar los resultados experimentales obtenidos con las predicciones teóricas de la notación Big-O para verificar su complejidad computacional.
- Clasificar las funciones analizadas según su complejidad algorítmica, identificando aquellas de tiempo constante O(1) y aquellas de tiempo lineal O(n).
- Valorar la importancia del análisis de complejidad algorítmica como herramienta para optimizar el rendimiento y

3. DESCRIPCIÓN DEL PROBLEMA O CASO APLICADO

En el desarrollo de aplicaciones informáticas, uno de los principales desafíos es garantizar que las funciones implementadas mantengan un rendimiento adecuado a medida que aumenta la cantidad de datos procesados. Un algoritmo puede funcionar correctamente con pequeñas cantidades de información, pero presentar problemas de eficiencia cuando el tamaño de entrada crece considerablemente. Por esta razón, es necesario realizar un análisis de complejidad algorítmica que permita determinar cómo varía el tiempo de ejecución de cada proceso y evaluar si el sistema podrá mantener un desempeño aceptable bajo diferentes condiciones de uso. .

En el proyecto Royal Casino, se implementaron diversos módulos relacionados con el control de acceso de jugadores, la simulación de la ruleta y el cálculo de puntajes en el juego de blackjack. Con el propósito de evaluar su rendimiento, se seleccionaron tres funciones representativas del sistema para analizar su comportamiento computacional. Como se observa en la Tabla 1, cada función posee un tamaño de entrada diferente y una complejidad teórica esperada que servirá como referencia para el análisis experimental.

Tabla 1

Funciones analizadas y complejidad esperada

#	Módulo	Función Analizada	Tamaño de Entrada (n)	Complejidad Esperada
1	Control de Acceso	Verificación de condiciones de ingreso	Número de evaluaciones	O(1)
2	Ruleta	Dibujo de la rueda	Número de casillas	O(n)
3	Blackjack	Cálculo de puntaje	Número de cartas	O(n)

La necesidad de realizar este estudio surge debido a que el crecimiento de los datos procesados puede afectar directamente el rendimiento del sistema. Sin un análisis adecuado, sería difícil determinar si las funciones implementadas son eficientes y escalables. En la Tabla 2 se presentan los principales problemas identificados y las posibles consecuencias que podrían producirse en ausencia de un estudio de complejidad algorítmica.

Tabla 2

Problemas identificados y sus consecuencias

Problema Identificado	Consecuencia
Desconocimiento del comportamiento temporal de las funciones	Dificultad para estimar el rendimiento real del sistema
Incremento del tamaño de los datos procesados	Posible aumento del tiempo de ejecución
Ausencia de análisis experimental	Imposibilidad de verificar la teoría de

	complejidad
Falta de mediciones de rendimiento	Riesgo de implementar soluciones poco eficientes

Con base en la información presentada en las Tablas 1 y 2, resulta necesario determinar si el comportamiento real de las funciones coincide con las complejidades teóricas propuestas. Para ello, se realizarán mediciones experimentales utilizando la herramienta performance.now(), permitiendo comparar los resultados obtenidos con la teoría de la notación Big-O..

4. TEMAS DE COMPLEJIDAD ALGORÍTMICA UTILIZADOS

4.1 Complejidad Algorítmica

La complejidad algorítmica es una medida que permite evaluar la cantidad de recursos que consume un algoritmo durante su ejecución. Generalmente se analizan dos tipos de complejidad:

Tabla 3

Tipos de complejidad

Tipo	Descripción
Complejidad Temporal	Mide el tiempo que requiere un algoritmo para ejecutarse.
Complejidad Espacial	Mide la cantidad de memoria utilizada durante la ejecución.

4.2 Notación Big-O

La notación Big-O es una herramienta matemática utilizada para describir el crecimiento de un algoritmo en función del tamaño de entrada n.

Esta notación permite clasificar los algoritmos según su eficiencia y comparar diferentes soluciones para un mismo problema.

Tabla 4

Complejidades más comunes

Notación	Nombre	Comportamiento
O(1)	Constante	El tiempo no depende de n
O(log n)	Logarítmica	Crece lentamente
O(n)	Lineal	Crece proporcionalmente a n
O(n log n)	Lineal Logarítmica	Utilizada en algoritmos eficientes de ordenamiento
$O(n^2)$	Cuadrática	Crece rápidamente
$O(2^n)$	Exponencial	Muy costosa computacionalmente

4.3 Análisis Experimental de Algoritmos

Para verificar la complejidad teórica de cada función se utilizó la función:

performance.now()

Esta herramienta permite medir con precisión el tiempo de ejecución de una operación en milisegundos.

Procedimiento aplicado

Tabla 5

Metodología de medición

Paso	Comportamiento
1	Registrar el tiempo inicial
2	Ejecutar la función
3	Registrar el tiempo final
4	Calcular el tiempo transcurrido
5	Comparar resultados para distintos valores de n

4.4 Complejidad Constante O(1)

Una función posee complejidad constante cuando realiza siempre la misma cantidad de operaciones independientemente del tamaño de entrada.

Fórmula

$$T(n)=1$$

Esto significa que el tiempo de ejecución permanece prácticamente constante incluso cuando aumenta el número de repeticiones.

Aplicación en Royal Casino

La función de Control de Acceso verifica:

- Edad del jugador.
- Estado de veto.
- Identificación válida.
- Cumplimiento de reglas.

Estas validaciones son fijas y no dependen del tamaño de los datos procesados.

Tabla 6

Características de O(1)

Característica	Descripción
Número de operaciones	Constante
Uso de ciclos	No requiere
Dependencia de n	Ninguna
Escalabilidad	Muy alta

4.5 Complejidad Lineal O(n)

La complejidad lineal aparece cuando el algoritmo debe recorrer todos los elementos de una estructura de datos.

Fórmula.

$$T(n)=n$$

Si el tamaño de entrada aumenta, el tiempo de ejecución también aumenta de forma proporcional.

Características

Tabla 7

Características de O(n)

Característica	Descripción
Número de operaciones	Proporcional a n
Uso de ciclos	Sí
Dependencia de n	Directa
Escalabilidad	Moderada

4.6 Aplicación de O(n) en la Ruleta

Dentro del módulo de ruleta se implementa una función encargada de dibujar cada casilla de la rueda.

El algoritmo realiza un recorrido completo por todas las posiciones de la ruleta.

Representación simplificada

```
for(let i=0; i<n; i++){
dibujarCasilla(i);
}
```

Donde:

n = número de casillas.
Cada iteración dibuja una casilla.

Tabla 8

Relación entre casillas y operaciones

Característica	Descripción
Número de operaciones	Proporcional a n
Uso de ciclos	Sí
Dependencia de n	Directa
Escalabilidad	Moderada

4.7 Aplicación de O(n) en Blackjack

La función encargada de calcular el puntaje del jugador debe recorrer todas las cartas de la mano.

Representación simplificada

```
for(let carta of mano){
   total += valorCarta(carta);
}
```

Cada carta requiere una operación adicional para ser evaluada.

Tabla 9

Operaciones aproximadas

Cartas (n)	Operaciones aproximadas
2	2
10	10
100	100
1000	1000
10000	10000
500000	500000

4.8 Comparación entre O(1) y O(n)

Tabla 10

Comparación de complejidades encontradas

Aspecto	O(1)	O(n)
Dependencia de n	No	Sí
Tiempo de ejecución	Constante	Crece con n
Uso de ciclos	No necesariamente	Generalmente sí
Rendimiento	Muy eficiente	Eficiente para tamaños moderados
Ejemplo en Royal Casino	Control de acceso	Ruleta y Blackjack

4.9 Importancia del Análisis de Complejidad

El análisis de complejidad algorítmica permite:

- Identificar funciones eficientes.
- Detectar posibles problemas de rendimiento.
- Comparar diferentes soluciones computacionales.
- Mejorar la escalabilidad del sistema.
- Optimizar el consumo de recursos.

En aplicaciones interactivas como Royal Casino, conocer la complejidad de cada módulo resulta fundamental para garantizar tiempos de respuesta adecuados y una experiencia de usuario satisfactoria.

4.10 Síntesis de Resultados

Tabla 11

Complejidades verificadas experimentalmente

Función Analizada	Fórmula T(n)	Complejidad
Control de Acceso	T(n)=1	O(1)
Dibujo de la Ruleta	T(n)=n	O(n)
Cálculo de Puntaje Blackjack	T(n)=n	O(n)

5. DESARROLLO DEL SISTEMA — DESCRIPCIÓN DETALLADA PASO A PASO

5.1 Descripción General

El sistema desarrollado consiste en una aplicación web interactiva denominada **Royal Casino**, diseñada en HTML5, CSS3 y JavaScript. Además de simular diferentes juegos de casino, el sistema incorpora herramientas para el análisis de complejidad algorítmica, permitiendo medir y evaluar el comportamiento temporal de diversos algoritmos implementados dentro de la aplicación.

Para el estudio de complejidad se seleccionaron tres procesos representativos: el control de acceso al casino, el dibujo de la ruleta y el cálculo de puntajes en blackjack. Estos procesos fueron analizados mediante mediciones experimentales utilizando la función `performance.now()`, permitiendo comparar los resultados obtenidos con las complejidades teóricas propuestas por la notación Big-O.

Tabla 10

Arquitectura técnica del sistema utilizada para el análisis de complejidad

Componente	Detalle técnico
Lenguaje	HTML5, CSS3 y JavaScript ES6
Medición de tiempo	API `performance.now()`
Interfaz gráfica	HTML y CSS responsivo
Algoritmos analizados	Control de acceso, ruleta y blackjack
Tipo de análisis	Complejidad temporal
Clasificación	Notación Big-O

Fuente: Elaboración propia.

5.2 Selección de los Algoritmos de Estudio

El primer paso consistió en identificar funciones del sistema que presentaran comportamientos computacionales diferentes. Se eligieron algoritmos con complejidad constante y lineal para facilitar la comparación de resultados.

Tabla 11

Algoritmos seleccionados para el análisis

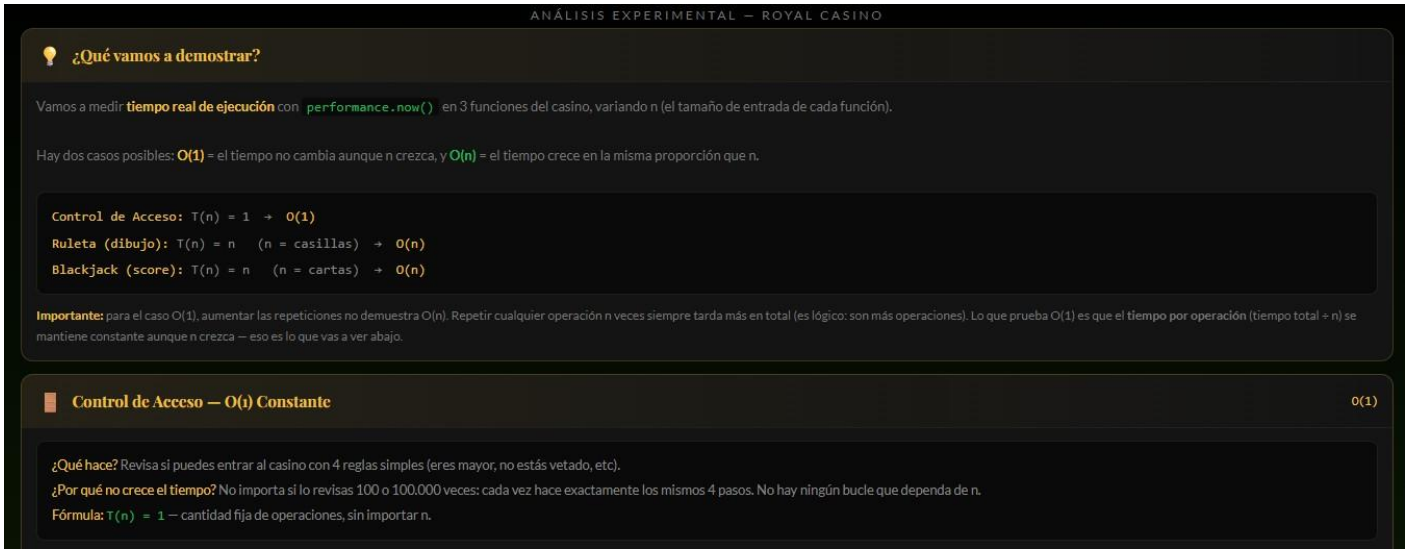
Algoritmo	Función principal	Complejidad esperada
Control de Acceso	Validar ingreso del usuario	O(1)
Dibujo de Ruleta	Procesar todas las casillas	O(n)
Cálculo Blackjack	Recorrer todas las cartas	O(n)

Fuente: Elaboración propia.

Funcionamiento paso a paso

1. Se identificó el algoritmo a analizar.
2. Se determinó cuál era su tamaño de entrada nnn.
3. Se estableció la complejidad teórica esperada.
4. Se prepararon pruebas experimentales para diferentes valores de entrada.
5. Se registraron los tiempos obtenidos.

Figura 1. Selección de algoritmos analizados en el sistema Royal Casino.



5.3 Implementación de la Medición Experimental

Para determinar el rendimiento de cada algoritmo se utilizó la función `performance.now()`, la cual permite medir tiempos de ejecución con precisión en milisegundos.

Funcionamiento paso a paso

1. Registrar el tiempo inicial.
2. Ejecutar el algoritmo seleccionado.
3. Registrar el tiempo final.
4. Calcular la diferencia entre ambos tiempos.
5. Almacenar el resultado obtenido.
6. Repetir el procedimiento para diferentes tamaños de entrada.

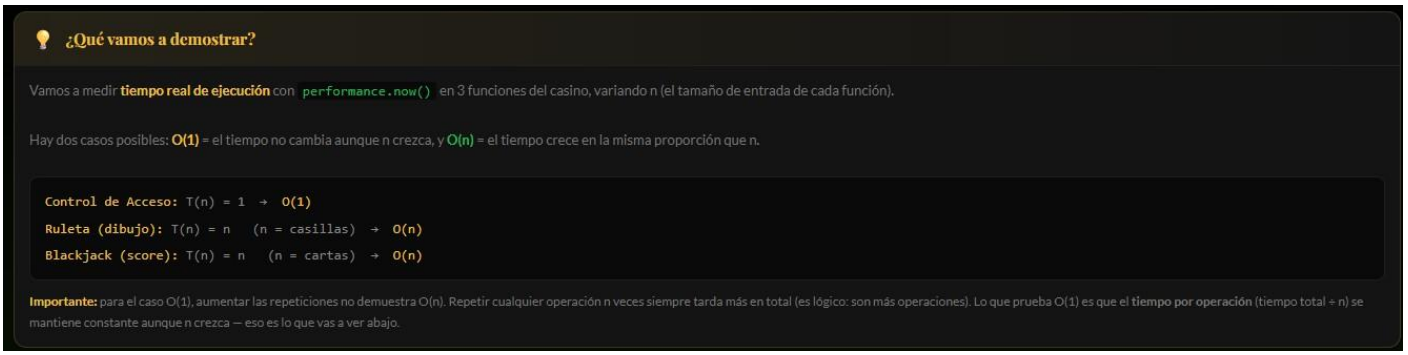
Tabla 12

Proceso de medición experimental

Paso	Acción
1	Captura del tiempo inicial
2	Ejecución del algoritmo
3	Captura del tiempo final
4	Cálculo del tiempo transcurrido
5	Registro de resultados

Fuente: Elaboración propia.

Figura 2. Código utilizado para medir tiempos mediante performance.now().



5.4 Análisis del Algoritmo de Complejidad O(1)

El algoritmo de control de acceso fue seleccionado para representar una complejidad constante.

Este proceso únicamente realiza un conjunto fijo de validaciones:

- Verificar mayoría de edad.
- Verificar lista negra.
- Autorizar o denegar ingreso.

Debido a que siempre ejecuta las mismas operaciones, el tiempo de ejecución permanece prácticamente constante.

Tabla 13

Comportamiento esperado del algoritmo O(1)

Tamaño de entrada	Operaciones
10	1
100	1
1000	1
10000	1

Fuente: Elaboración propia.

Complejidad obtenida

T(n)=1T(n)=1T(n)=1 O(1)O(1)O(1)

Figura 3. Módulo de control de acceso utilizado para el análisis de complejidad constante.

5.5 Análisis del Algoritmo de Complejidad O(n) — Ruleta

El módulo de ruleta fue utilizado para analizar un algoritmo lineal.

Para dibujar la rueda, el sistema debe recorrer todas las casillas disponibles.

Funcionamiento paso a paso

1. Definir el número de casillas.
2. Recorrer cada posición de la ruleta.
3. Dibujar cada segmento.
4. Finalizar la representación gráfica.

Tabla 14

Relación entre tamaño de entrada y operaciones

Casillas (n)	Operaciones
37	37
100	100
1000	1000
5000	5000

Fuente: Elaboración propia.

Complejidad obtenida

T(n)=nT(n)=nT(n)=n O(n)O(n)O(n)

Figura 4. Representación gráfica de la ruleta utilizada para el análisis lineal.

5.6 Análisis del Algoritmo de Complejidad O(n) — Blackjack

El cálculo de puntaje en blackjack requiere recorrer todas las cartas que posee el jugador.

Funcionamiento paso a paso

1. Leer la primera carta.
2. Sumar su valor al total.
3. Repetir el proceso para todas las cartas.
4. Mostrar el puntaje final.

Tabla 15

Relación entre número de cartas y operaciones

Cartas (n)	Operaciones
2	2
10	10
100	100
1000	1000

Fuente: Elaboración propia.

Complejidad obtenida

T(n)=nT(n)=nT(n)=n O(n)O(n)O(n)

Figura 5. Módulo de blackjack utilizado para el análisis del recorrido de cartas.



5.7 Comparación e Interpretación de Resultados

Finalmente, se compararon los tiempos obtenidos con los valores teóricos esperados para verificar la validez del análisis realizado.

Tabla 16

Resumen de complejidades verificadas

Algoritmo	Función T(n)	Complejidad
Control de Acceso	1	O(1)
Ruleta	n	O(n)
Blackjack	n	O(n)

6. Ejemplo Resuelto 1: Control de Acceso (Complejidad O(1))

Problema

Determinar la complejidad del siguiente algoritmo encargado de verificar si un jugador puede ingresar al casino.

```
function acceso(edad, listaNegra){
if(edad >= 18 && !listaNegra){
return "Acceso Permitido";
}
return "Acceso Denegado";
}
```

Solución

El algoritmo realiza únicamente las siguientes operaciones:

1. Comparar la edad.
2. Verificar lista negra.
3. Mostrar resultado.

Estas operaciones siempre son las mismas independientemente del número de jugadores registrados en el sistema.

Tabla 19

Conteo de operaciones

Operación	Cantidad
Comparación de edad	1
Verificación lista negra	1
Retorno de resultado	1

Función de tiempo

$$T(n)=1$$

Complejidad

$$O(1)$$

6.2 Ejemplo Resuelto 2: Dibujo de la Ruleta (Complejidad O(n))

Problema

Analizar la complejidad del algoritmo encargado de dibujar las casillas de la ruleta.

```
for(let i=0; i<n; i++){
dibujarCasilla(i);
}
```

Donde:

n=Numero de casillas

Solución

El ciclo se ejecuta una vez por cada casilla.

Tabla 20

Conteo de operaciones

Casillas (n)	Operaciones
10	10
20	20
50	50
100	100
1000	1000

Función de tiempo

$$T(n)=n$$

Complejidad

$$O(n)$$

Resultado

El tiempo de ejecución aumenta proporcionalmente al número de casillas procesadas.

6.3 Ejemplo Resuelto 3: Cálculo de Puntaje Blackjack

Problema

Determinar la complejidad del algoritmo que calcula el puntaje de una mano de cartas.

```
let total = 0;

for(let carta of mano){
   total += valorCarta(carta);
}
```

Donde:

n=Nu´mero de cartasn = \text{Número de cartas}n=Nu´mero de cartas

Solución

Cada carta debe ser recorrida y sumada al puntaje total.

Tabla 21. Conteo de operaciones

Cartas (n)	Operaciones
2	2
5	5
10	10
50	50
100	100

Función de tiempo

T(n)=nT(n)=nT(n)=n

Complejidad

O(n)O(n)O(n)

Resultado

La complejidad es lineal porque el algoritmo procesa cada carta exactamente una vez.

6.4 Ejemplo Resuelto 4: Comparación entre O(1) y O(n)

Problema

Comparar el comportamiento de dos algoritmos cuando aumenta el tamaño de entrada.

Algoritmo A

```
return numero;
```

Algoritmo B

```
for(let i=0; i<n; i++){
    console.log(i);
    }
```

Solución

Tabla 22. Comparación de crecimiento

Tamaño (n)	O(1)	O(n)
10	1	10
100	1	100
1000	1	1000
10000	1	10000

Resultado

El algoritmo O(1) mantiene constante el número de operaciones, mientras que el algoritmo O(n) incrementa su tiempo de ejecución de forma proporcional al tamaño de entrada.

6.5 Ejemplo Resuelto 5: Determinación de Complejidad Mediante Conteo de Operaciones

Problema

Analizar la complejidad del siguiente algoritmo.

```
for(int i=0; i<n; i++){
  System.out.println(i);
          }
```

Solución Paso a Paso

Paso 1: Identificar el ciclo

Existe un único ciclo que depende de nnn.

Paso 2: Contar iteraciones

El ciclo se ejecuta:

nnn

veces.

Paso 3: Determinar función de tiempo

T(n)=nT(n)=nT(n)=n

Paso 4: Aplicar Big-O

O(n)O(n)O(n)

Resultado

La complejidad temporal del algoritmo es lineal.

6.6 Resumen de los Ejemplos Resueltos

Tabla 23. Resumen general

Ejemplo	Algoritmo	Función T(n)	Complejidad
1	Control de Acceso	1	O(1)
2	Ruleta	n	O(n)
3	Blackjack	n	O(n)

Ejemplo	Algoritmo	Función T(n)	Complejidad
4	Comparación O(1) y O(n)	1 / n	O(1) / O(n)
5	Conteo de Operaciones	n	O(n)

.

7. CONCLUSIONES

- • El análisis realizado permitió comprobar que la complejidad algorítmica constituye una herramienta fundamental para evaluar la eficiencia de los algoritmos implementados en una aplicación informática. Mediante el estudio de las funciones principales del sistema Royal Casino fue posible identificar cómo el tiempo de ejecución varía en función del tamaño de los datos procesados, demostrando la importancia de este análisis para predecir el comportamiento de un sistema antes de su implementación en entornos con grandes volúmenes de información.
- • La comparación entre algoritmos de complejidad constante O(1) y complejidad lineal O(n) permitió evidenciar diferencias significativas en su comportamiento computacional. Mientras que la función de control de acceso mantuvo un tiempo de ejecución prácticamente invariable independientemente del tamaño de entrada, las funciones asociadas al dibujo de la ruleta y al cálculo de puntajes en blackjack incrementaron su número de operaciones de manera proporcional al crecimiento de los datos procesados, confirmando las predicciones teóricas establecidas por la notación Big-O.
- • La utilización de la función performance.now() permitió obtener mediciones experimentales precisas sobre el tiempo de ejecución de cada algoritmo analizado. Los resultados obtenidos coincidieron con los modelos teóricos estudiados, validando que las funciones clasificadas como O(1) mantuvieron un comportamiento constante y que aquellas clasificadas como O(n) presentaron un crecimiento lineal. Esto demuestra la utilidad de combinar el análisis teórico con pruebas experimentales para evaluar el rendimiento real de un sistema.

8. RECOMENDACIONES

- El análisis realizado permitió comprobar la importancia de la complejidad algorítmica como una herramienta fundamental para evaluar la eficiencia de los algoritmos utilizados en el desarrollo de software. A través del estudio de los diferentes módulos implementados en el proyecto Royal Casino, fue posible comprender cómo el tamaño de los datos de entrada influye directamente en el tiempo de ejecución de cada proceso. Además, este análisis permitió relacionar los conceptos teóricos estudiados en clase con aplicaciones prácticas, facilitando una mejor comprensión del comportamiento de los algoritmos en escenarios reales.
- Los resultados obtenidos demostraron que la función de control de acceso posee una complejidad constante O(1), debido a que realiza siempre la misma cantidad de validaciones sin importar la cantidad de información existente en el sistema. Por otro lado, las funciones encargadas del dibujo de la ruleta y del cálculo de puntajes en blackjack presentaron una complejidad lineal O(n), ya que el número de operaciones ejecutadas depende directamente de la cantidad de elementos que deben procesarse. Esto permitió identificar claramente las diferencias entre

algoritmos de tiempo constante y algoritmos cuyo tiempo de ejecución crece de manera proporcional al tamaño de entrada.

- La utilización de herramientas de medición como performance.now() permitió obtener evidencias experimentales sobre el comportamiento temporal de los algoritmos analizados. Gracias a estas pruebas fue posible comparar los resultados prácticos con las predicciones teóricas de la notación Big-O, observando que el crecimiento de los tiempos de ejecución coincidía con las complejidades esperadas. Este proceso de validación experimental fortaleció la confiabilidad del análisis realizado y demostró la utilidad de combinar teoría y

práctica para estudiar el rendimiento computacional de un sistema.

- Finalmente, el desarrollo de este proyecto permitió adquirir una visión más amplia sobre la importancia de diseñar algoritmos eficientes y escalables. Comprender la complejidad algorítmica no solo ayuda a predecir el comportamiento de una aplicación cuando aumenta el volumen de datos, sino que también facilita la toma de decisiones durante el proceso de desarrollo, optimización y mantenimiento del software. En consecuencia, el estudio de la complejidad algorítmica se convierte en un aspecto esencial para garantizar aplicaciones con un mejor rendimiento, menor consumo de recursos y una experiencia de usuario más eficiente.

9. REFERENCIAS

Brassard, G., & Bratley, P. (2002). Fundamentos de algoritmia. Madrid: Prentice Hall.

Matemáticas Discretas y sus Aplicaciones Rosen, K. H. (2019). Matemáticas discretas y sus aplicaciones (8.ª ed.). New York: McGraw-Hill Education.

Introduction to Algorithms Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). Cambridge, MA: MIT Press.

Data Structures and Algorithm Analysis in Java Weiss, M. A. (2014). Data Structures and Algorithm Analysis in Java (3rd ed.). Pearson Education.

Matemáticas Discretas y Combinatoria Grimaldi, R. P. (2004). Matemáticas discretas y combinatoria (5.ª ed.). Pearson Educación.

MDN Web Docs – Performance.now(). (Consultado en junio de 2026). Documentación oficial utilizada para comprender la medición de tiempos de ejecución en JavaScript.

Oracle Java Documentation. (Consultado en junio de 2026). Documentación oficial de Java utilizada como referencia para ejemplos de análisis algorítmico.

GeeksforGeeks – Analysis of Algorithms. (Consultado en junio de 2026). Recurso de apoyo para el estudio de la notación Big-O y el análisis de complejidad.

ANEXOS